Otomasyon ve Hatalardan Öğrenme

ÖFY

October 2026

Özet

Bir süreci otomatikleştirmek, o süreçteki hataları görmenin güçlü bir yoludur. Bunun sebebi otomasyonun kendisi değil, otomasyonun zorunlu kıldığı üç özelliktir: açıklık, tekrarlanabilirlik ve ölçülebilirlik. Buna karşın "otomatikleştirilemeyen bir sürecin hataları öğrenilemez" iddiası fazla güçlüdür. İnsanlar otomasyon olmadan da hatalarından öğrenir; otomasyon ise yanlış kurulduğunda hatayı gizleyip büyütebilir. Otomasyonun hatalara karşı nasıl kullanılacağını hatanın kaynağı belirler. Mantıktan gelen hata bir kez düzeltilir ve testle kilitlenir. Girdiden gelen hata önceden kapatılamaz; her çıktıda ölçülür, işaretlenir ve kararı insan verir. Aynı belirti birden çok kaynaktan çıkabildiği için iki yöntem çoğu zaman birlikte uygulanır. Hatanın ciddiyeti yöntemi değil, eşiğin sıkılığını belirler.

Giriş

Yazılım geliştirmede yaygın bir sezgi vardır: bir iş elle yapıldığı sürece hatalarından sistemli biçimde ders çıkarılamaz. Bu sezgi bazen "otomatikleştiremezsen kendindeki hataları öğrenemezsin" biçiminde ifade edilir. İfadenin bir doğru çekirdeği vardır, ancak bu hâliyle hem fazla geniş hem de eksiktir. Fazla geniştir, çünkü otomasyon dışındaki öğrenme yollarını yok sayar. Eksiktir, çünkü farklı türdeki hataların farklı araçlar gerektirdiğini söylemez.

Gövdede kullanılan temel terimler şunlardır:

- **Otomasyon**: Bir işin adımlarının, insan müdahalesi olmadan bir makine tarafından aynı biçimde tekrar edilecek şekilde tarif edilmesi.
- **Mantık hatası**: Kodun ya da sürecin kendi kurgusundan doğan hata. Aynı girdi her seferinde aynı hatayı üretir.
- **Girdi hatası**: Kurgu doğru olduğu hâlde, sürece gelen verinin beklenenden farklı olmasından doğan hata. Her seferinde farklı biçimde ortaya çıkabilir.
- **Belirti**: Hatanın dışarıdan görünen sonucu; örneğin boş bir çıktı ya da yanlış bir değer.
- **Sebep**: Belirtiyi doğuran asıl kusur.
- **Regresyon testi** (*regression test*): Düzeltilmiş bir hatanın ya da verilmiş bir kararın, sonraki değişikliklerle geri gelmediğini doğrulayan otomatik test.
- **Çıktı kontrolü**: Sistem çalışırken her üretilen sonucu belirli ölçütlere göre denetleyen ve sorunlu olanı işaretleyen mekanizma.

Not iki soruyu kapsar. Birincisi, otomasyon ile hatalardan öğrenme arasındaki ilişkinin sınırlarıdır. İkincisi, hatanın kaynağına göre hangi önleme yönteminin seçileceğidir. Notun kapsamı yazılım ve tekrarlanan iş süreçleridir; insan ilişkileri ya da sanat gibi otomatikleşmeyen alanlar yalnızca sınır örneği olarak ele alınır.

Otomasyon ve Hatalardan Öğrenme

Otomasyon hatayı kendiliğinden öğretmez. Hatayı görünür kılan, otomasyonun sürece dayattığı koşullardır. Bu koşullar sağlanmadan kurulan otomasyon, hatayı görünür kılmak yerine gizleyebilir.

İddianın Doğru Olan Çekirdeği

Açıklık

Bir işi otomatikleştirmek için her adımın açıkça tarif edilmesi gerekir. Elle yapılan bir işte birçok karar fark edilmeden, sezgiyle verilir. Bu kararlar yazıya dökülmeye çalışıldığında, aslında net olarak bilinmeyen adımlar ortaya çıkar. Örneğin bir raporu her ay elle hazırlayan biri, "eksik satırları ne yapıyorum" sorusunu hiç sormamış olabilir. Raporu otomatikleştirmeye çalıştığında bu soruya bir cevap yazmak zorunda kalır. Boşluk, otomasyon yazılırken görünür olur.

Tekrarlanabilirlik

İnsan aynı işi her seferinde biraz farklı yapar. Bu yüzden bir hata çıktığında, sebebin süreçte mi yoksa o günkü uygulamada mı olduğu belirsiz kalır. Makine ise her seferinde aynı adımları uygular. Otomatik bir süreçte hata çıktığında, kaynağın sürecin kendisinde ya da gelen veride olduğu bellidir; uygulayıcıdan gelen değişkenlik ortadan kalkmıştır.

Ölçülebilirlik

Otomatik süreçler test, log ve metrik üretebilir. Bunlar hatanın ne zaman, nerede ve ne sıklıkla ortaya çıktığını kayda geçirir. Kayda geçmeyen hata fark edilmez; fark edilmeyen hata da düzeltilemez.

İddianın Aşırı Güçlü Olan Yönü

Otomasyonsuz Öğrenme

İnsanlar otomasyon olmadan da hatalarından öğrenir. Geri bildirim almak, not tutmak, bir işi sonradan gözden geçirmek ya da deneyimli birinden eleştiri dinlemek de öğrenme yollarıdır. Bu yolların ortak noktası, otomasyonda olduğu gibi, hatayı görünür ve tarif edilebilir hâle getirmeleridir. Otomasyon bu yollardan yalnızca biridir.

Otomatikleşmeyen Alanlar

Karar verme, yazı yazma, insan ilişkileri ve sanat gibi alanlar otomatikleştirilemez. Bu alanlarda da hatalar fark edilir ve düzeltilir. Bir yazar eski metnini okuyup aynı anlatım hatasını tekrarlamamayı öğrenebilir. İddia bu alanlara uygulandığında açıkça yanlış sonuç verir.

Otomasyonun Hatayı Gizlemesi

Otomasyon yanlış bir süreç üzerine kurulursa, hatayı ortadan kaldırmaz; daha hızlı ve daha büyük ölçekte tekrarlar. Testler de bu sorunu tek başına çözmez. Bir test yalnızca kendisi için tanımlanmış durumu denetler. Dijkstra (1970), testin hatanın varlığını gösterebileceğini ama yokluğunu kanıtlayamayacağını vurgulamıştır. Tüm testlerin başarılı olması, tanımlanmamış bir hatanın bulunmadığı anlamına gelmez.

Bainbridge (1983) benzer bir sorunu endüstriyel otomasyon için tarif etmiştir: süreç otomatikleştikçe insan operatörün sistemi anlama ve olağan dışı durumu fark etme becerisi zayıflar. Otomasyon, dikkat edilmezse, hatayı insanın gözünden uzaklaştırır.

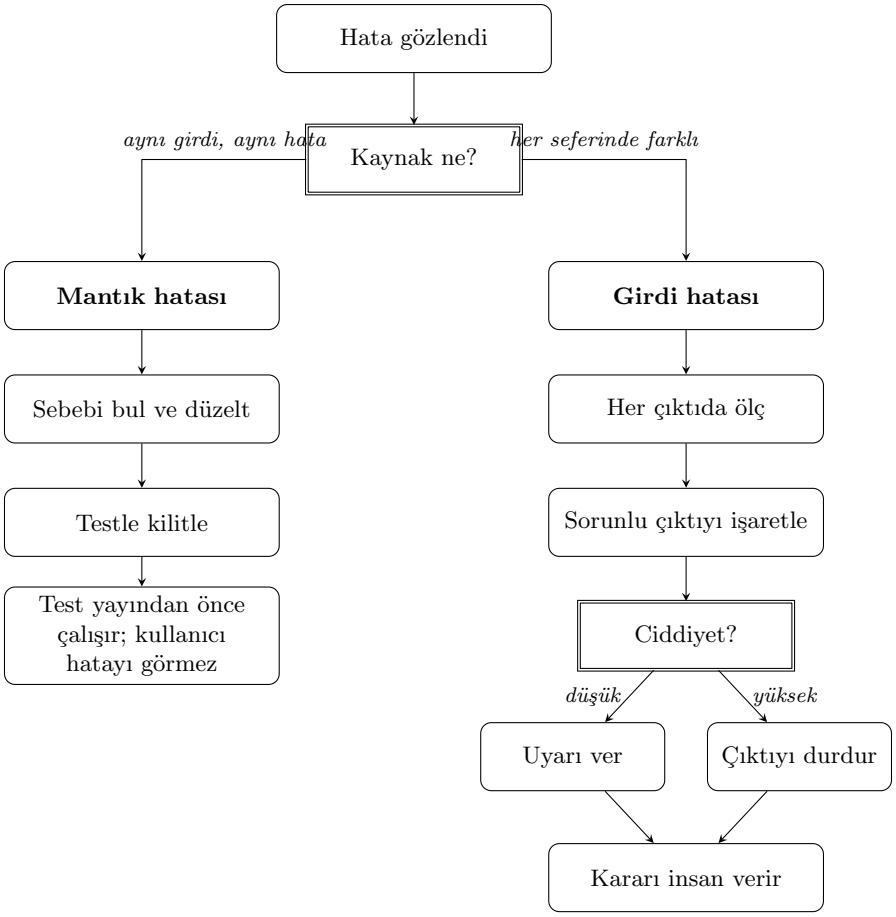
Savunulabilir İfade

İddianın savunulabilir hâli şu iki cümleyle yazılabilir:

- Açıkça tarif edilemeyen bir süreçteki hatalar sistemli biçimde öğrenilemez.
- Bir süreci otomatikleştirmeye çalışmak, o süreçteki hataları görmenin en etkili yollarından biridir.

Bu ifadelerde ağırlık otomasyonun kendisinde değil, otomasyonun zorunlu kıldığı açıklık, tekrarlanabilirlik ve ölçülebilirliktedir.

Hatanın Kaynağına Göre Önleme



Hatanın kaynağı aracı, ciddiyeti eşiğin sıkılığını belirler.

Otomasyonun hatalara karşı nasıl kullanılacağı, hatanın nereden geldiğine bağlıdır. Yanlış araç seçildiğinde ya hata tekrar eder ya da gereksiz bir yük oluşur.

Kaynak Sorusu

Bir hata ya sürecin mantığından ya da sürece gelen girdiden doğar. Mantık her seferinde aynı biçimde çalışır; girdi ise her seferinde farklıdır. Hatanın hangisinden geldiği, nasıl önleneceğini belirler.

Gray (1985) yazılım hatalarını tekrar üretilebilirliklerine göre ayırmıştır: aynı koşulda her seferinde ortaya çıkan hatalar ile yalnızca bazı koşullarda görünen hatalar. Buradaki mantık–girdi ayrımı bununla aynı değildir, ama benzer bir sezgiye dayanır: tekrar üretilebilen hata ile her seferinde farklı ortaya

çıkan hata farklı araçlar ister.

Mantık Hatası: Düzelt ve Testle Kilitle

Mantık hatasında aynı girdi her zaman aynı hatayı verir. Bu yüzden sebep bulunabilir, düzeltilebilir ve bir testle kilitlenebilir. Test, yayına çıkmadan önce çalışır; kullanıcı hatayı hiç görmez.

Örnek olarak, tarihleri ayrıştıran bir fonksiyon "31.12.2025" biçimini her seferinde yanlış okuyor olsun. Bu bir mantık hatasıdır. Fonksiyon bir kez düzeltilir ve bu tarihi içeren bir test eklenir. Sonraki bir değişiklik aynı hatayı geri getirirse test başarısız olur ve sorun yayından önce yakalanır.

Girdi Hatası: Ölç ve İşaretle

Mantık doğru olsa bile girdi sorun çıkarabilir. Bozuk kodlamalı bir dosya, eksik bir alan ya da beklenmedik bir biçim önceden bilinemez. Bu tür hatalar test aşamasında kapatılamaz, çünkü hangi girdinin geleceği önceden belli değildir.

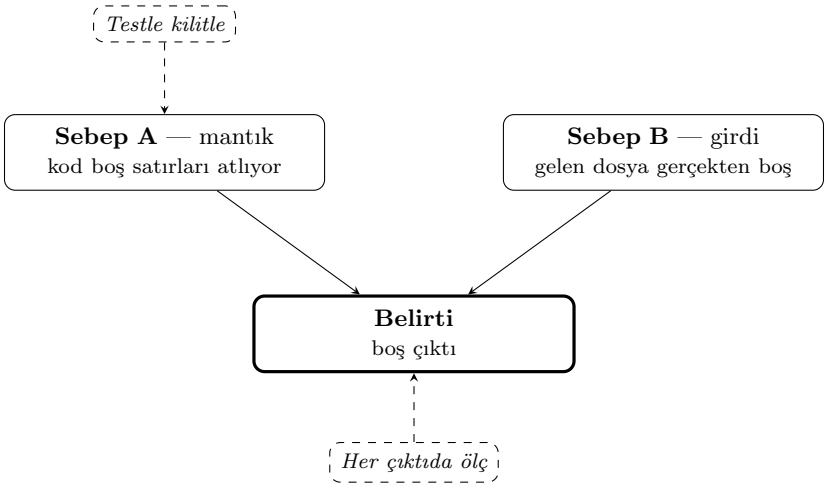
Bu durumda her çıktı bir kontrolden geçirilir. Kontrol hatayı düzeltmez; yalnızca sorunlu çıktıyı işaretler. Ne yapılacağına insan karar verir. Örneğin bir dosya içe aktarma sürecinde, zorunlu bir alanı boş olan kayıtlar işaretlenir ve bir kişi bu kayıtları inceleyip düzeltir ya da reddeder.

Ayırt Etme Ölçütü: Düzeltmenin Yeri

Bir hatanın mantıktan mı girdiden mi geldiğini anlamanın pratik yolu, düzeltmenin nerede yapıldığına bakmaktır:

- Kod bir kez değiştirildiğinde sorun her durumda ortadan kalkıyorsa, hata mantık hatasıdır.
- Sorunu gidermek için her seferinde gelen veriye bakmak gerekiyorsa, hata girdi hatasıdır.

Belirti ve Sebep Ayrımı



Sebep A düzeltilse bile aynı belirti Sebep B yoluyla yeniden görülebilir.

Bir sebep düzeltildiğinde o sebep ortadan kalkar. Ancak aynı belirti başka sebeplerden de doğabilir. Örneğin boş bir çıktı iki farklı yoldan oluşabilir: kod boş satırları yanlışlıkla atlıyor olabilir (mantık hatası) ya da gelen dosya gerçekten boş olabilir (girdi hatası). Birinci sebep düzeltilip testle kilitlense de, ikinci sebep yüzünden aynı belirti yeniden görülebilir.

Bu durumda iki yöntem birlikte uygulanır: bilinen sebep testle kilitlenir, belirti ise her çıktıda ölçülür.

Kararların Kilitlenmesi

Testler yalnızca hataları değil, verilmiş kararları da korur. Yanlış olduğu anlaşılan bir kural kaldırıldığında, ileride biri tarafından iyi niyetle geri eklenmemesi için o karar da testle korunur.

Örnek olarak, bir sistemin e-posta adreslerinde artı işaretini (+) reddettiği ve bu kuralın yanlış olduğu anlaşılıp kaldırıldığı düşünülsün. Artı işaretli bir adresin kabul edildiğini doğrulayan bir test eklenirse, kural geri getirildiğinde test başarısız olur. Böylece test, kararın nedenini bilmeyen birine bile o kararı hatırlatır.

Ciddiyet ve Sıkılık

Ciddiyet (sıklığı seçer)	Hatanın kaynağı (aracı seçer): Mantık	Hatanın kaynağı (aracı seçer): Girdi
Yüksek	Düzelt ve testle kilitle; test geçmezse yayın durur	Her çıktıda ölç; çıktıyı durdur, insan onayı iste
Düşük	Düzelt ve testle kilitle	Her çıktıda ölç; uyarı ver

Sütun yöntemi, satır sıkılığı belirler.

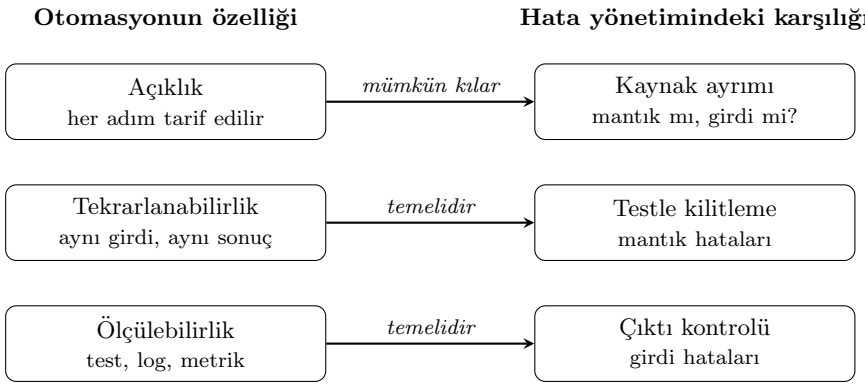
Hangi aracın kullanılacağını hatanın kaynağı belirler; ne kadar sıkı olunacağını ise hatanın ciddiyeti belirler. Bu iki boyut birbirinden ayrıdır.

Girdi kontrolü örneğinde, isteğe bağlı bir alanın eksik olması önemsiz bir sorundur ve bir uyarı yeterlidir. Zorunlu bir alanın eksik olması ise ciddi bir sorundur; bu durumda çıktı durdurulur ve insan onayı olmadan devam edilmez.

Karar Tablosu

Durum	Yöntem
Hata mantıktan geliyor	Düzelt ve testle kilitle
Hata girdiden geliyor	Her çıktıda ölç ve işaretle
Aynı belirti birden çok yoldan geliyor	Sebebi testle kilitle, belirtiyi her çıktıda ölç
Yanlış bir kural kaldırıldı	Kararı testle koru
Hata ciddi	Eşiği sıkılaştır; uyarı yerine çıktıyı durdur

İki Çerçevenin Birleşimi



Otomasyonun hatayı görünür kılan özellikleri, önleme yöntemlerinin ön koşuludur.

Otomasyonun hatayı görünür kılan üç özelliği, hata önleme yöntemleriyle doğrudan eşleşir.

- **Açıklık**, hatanın kaynağını ayırt etmeyi mümkün kılar. Süreç açıkça tarif edilmemişse, bir hatanın mantıktan mı girdiden mi geldiği sorulamaz bile.
- **Tekrarlanabilirlik**, mantık hatalarının testle kilitlenmesinin temelidir. Aynı girdinin aynı sonucu vermesi, bir testin anlamlı olmasının ön koşuludur.
- **Ölçülebilirlik**, girdi hatalarının yakalanmasının temelidir. Her çıktının kontrol edilebilmesi, sürecin ölçüm üretmesine bağlıdır.

Otomasyonun sınırları da bu yöntemlerde karşılık bulur. Testlerin yalnızca tanımlı durumları yakalaması, girdi hatalarının neden testle değil ölçümle ele alındığını açıklar. Bainbridge'in (1983) tarif ettiği, insanın otomatik sistemden uzaklaşması sorunu ise girdi kontrolünde kararın insana bırakılmasıyla dengelenir. Kontrol işaretler, insan karar verir.

Sonuç

Otomasyon ile hatalardan öğrenme arasındaki ilişki zorunluluk değil, kolaylaştırma ilişkisidir. Otomasyon olmadan da öğrenilir, ancak otomasyonun dayattığı açıklık, tekrarlanabilirlik ve ölçülebilirlik öğrenmeyi sistemli hâle getirir. Bu yüzden "otomatikleştiremezsen öğrenemezsin" yerine "tarif edemezsen sistemli öğrenemezsin" demek daha doğrudur.

Bu çerçevenin pratik karşılığı, her hata karşısında önce kaynağın sorulmasıdır. Mantık hatası için yatırım bir kez yapılır ve kalıcıdır. Girdi hatası için yatırım süreklidir; her çıktıda kontrol ve gerektiğinde insan kararı gerekir. Bu ayrım yapılmadan seçilen araç ya hatayı tekrar ettirir ya da gereksiz yük üretir.

Çerçevenin sınırları da vardır. Mantık ile girdi arasındaki çizgi her zaman net değildir. Örneğin belirli bir girdi türünü hiç hesaba katmamış bir kod, hem mantık hem girdi hatası olarak okunabilir; bu durumda girdi türü öğrenildikçe mantığa eklenir ve hata sınıf değiştirir. Ayrıca testle kilitlenen kararlar zamanla eskiyebilir. Bir kararı koruyan test, o kararın artık yanlış olduğu durumda değişime direnç oluşturabilir. Bu nedenle testlerin hangi kararı neden koruduğunun kaydının tutulması gerekir.

Açık kalan soru, otomatikleşmeyen alanlarda aynı ayrımın ne ölçüde uygulanabileceğidir. Yazı ya da karar verme gibi alanlarda da tekrar eden hatalar (mantık) ile duruma özgü hatalar (girdi) ayırt edilebilir. Ancak bu alanlarda "testle kilitleme" karşılığının ne olduğu, örneğin kontrol listeleri ya da gözden geçirme alışkanlıkları, ayrıca incelenmeye değerdir.

Kaynaklar

- Bainbridge, L. (1983). Ironies of Automation. *Automatica*, 19(6).
- Dijkstra, E. W. (1970). *Notes on Structured Programming* (EWD249).
- Gray, J. (1985). *Why Do Computers Stop and What Can Be Done About It?* Tandem Computers Technical Report 85.7.